

Campus Event Manager

Rapport technique — Projet NoSQL Development

Cohorte : MCS DE1 · Master 1 Data Engineering & Cloud Computing

Auteur : ZOUI El Mahdi

Base de données : MongoDB

Stack : Node.js · Express · EJS · Bootstrap 5

Date : 11 septembre 2026

1. Vue d'ensemble du projet

L'application **Campus Event Manager** permet à une université de gérer ses événements de campus (workshops, talks, meetups, hackathons, activités étudiantes) et le suivi des inscriptions étudiantes. L'objectif du projet est de démontrer la conception et l'exploitation d'une base **NoSQL orientée document** avec MongoDB, incluant l'utilisation combinée d'embedding, de références, de mise à jour de tableaux imbriqués et de pipelines d'agrégation.

Stack retenue : Node.js + Express + driver MongoDB officiel + EJS/Bootstrap. Cette stack privilégie la rapidité d'exécution et l'accès direct à la syntaxe MongoDB (les pipelines ne sont pas cachés derrière un ODM).

MongoDB est hébergé sur une VM Ubuntu (nom d'hôte `emzoui`) et joint via la variable d'environnement `MONGODB_URI`. La base est nommée `campus_events`.

2. Modèle de données NoSQL

2.1 Collection `users`

```
{
  "_id": ObjectId,
  "firstName": "Amine",
  "lastName": "Zouiten",
  "email": "amine.zouiten@campus.edu",
  "department": "Data",
  "role": "student",
  "interests": ["nosql", "python", "ml"],
  "createdAt": ISODate
}
```

- `interests` : tableau de strings.
- `email` : indexé unique.
- `role` : énumération contrôlée (*student, teacher, staff, admin*).

2.2 Collection `events`

```
{
  "_id": ObjectId,
  "title": "MongoDB Aggregation Workshop",
  "description": "...",
  "category": "Workshop",
  "tags": ["nosql", "mongodb", "data"],
  "startDate": ISODate,
  "endDate": ISODate,
  "capacity": 25,
  "location": {
    "building": "Innovation Center",
    "room": "B204",
  }
}
```

```

    "campus": "Paris"
  },
  "organizerId": ObjectId,
  "registrations": [
    { "userId": ObjectId, "registeredAt": ISODate, "status": "confirmed" }
  ],
  "createdAt": ISODate
}

```

- location : **objet imbriqué (embedded)**.
- tags : tableau de strings, indexé (multikey).
- registrations : **tableau d'objets embedded**.
- organizerId, registrations.userId : **références** vers users._id.

2.3 Indexes

Index	Type	Justification
users.email	unique	Contrainte métier, recherche par email.
events.startDate	ascending	Tri du catalogue par date, dashboard "prochains events".
events.category	ascending	Filtre côté catalogue et pipeline "par catégorie".
events.tags	multikey	Filtre par tag.
events.registrations.userId	multikey	Recherche des events auxquels un user est inscrit.

2.4 Validation JSON-Schema

Appliquée côté MongoDB (\$jsonSchema dans createCollection) pour garantir :

- title non vide, capacity ≥ 1 , dates typées, structure location complète ;
- role et registrations.status restreints à des valeurs contrôlées ;
- email conforme à un pattern regex.

Une validation applicative complète la validation Mongo (messages d'erreur lisibles en UI), et vérifie notamment que endDate $\geq$ startDate avant chaque insert/update.

3. Choix de modélisation : embedding vs referencing

3.1 Les inscriptions sont *embedded* dans events.registrations

Chaque event a un nombre borné de participants (quelques dizaines à ~200 max), et les inscriptions sont **toujours lues avec l'event** (page détail, dashboard). Un tableau embedded permet des mises à jour atomiques via \$push/\$pull/\$ positional et un accès à \$size sans join. Une collection séparée aurait multiplié les requêtes sans bénéfice.

3.2 L'organisateur et les utilisateurs sont référencés par ObjectId

Un utilisateur peut être organisateur ou participant de nombreux events ; ses données (email, département...) doivent rester cohérentes en un seul endroit. Dupliquer les infos dans chaque event aurait conduit à une dénormalisation coûteuse à maintenir. Le prix payé — un \$lookup ou

un `find({_id: {$in:[...]}})` — reste faible car les volumes sont raisonnables et les indexes couvrent la jointure.

4. Architecture de l'application

```
src/
├── app.js                # bootstrap Express + EJS + method-override
├── db.js                 # singleton MongoClient (driver v6)
├── routes/               # handlers HTTP par domaine
│   ├── dashboard.js
│   ├── events.js
│   ├── users.js
│   └── analytics.js
├── repositories/        # TOUTES les opérations MongoDB
│   ├── events.js        # CRUD + agrégations + registrations
│   ├── users.js         # CRUD + $lookup + users sans inscription
│   └── analytics.js     # pipelines A..F
├── views/               # templates EJS (5 pages)
└── public/css/style.css
```

La séparation **routes** ↔ **repositories** rend les requêtes et pipelines MongoDB immédiatement identifiables dans le code, comme demandé dans le brief.

5. Pages de l'application

Page	Route	Fonctions clés (côté MongoDB)
Dashboard	/	<code>countDocuments</code> , <code>find + sort + limit</code> , agrégation <code>\$unwind + \$match + \$group</code> pour le total confirmé, <code>\$filter + \$size</code> pour le plus populaire.
Events	/events	CRUD complet (<code>insertOne</code> , <code>updateOne</code> , <code>deleteOne</code>), filtres via <code>\$regex</code> , <code>\$gte/\$lt</code> , tag array, tri, projection.
Détail event	/events/:id	<code>\$push</code> (nouvelle inscription), positional <code>\$</code> (<code>\$set</code> sur <code>registrations.\$.status</code>) pour annuler, <code>\$pull</code> pour retirer, contrôle capacité + anti-doublon.
Users	/users	CRUD, <code>\$lookup</code> avec pipeline pour compter les inscriptions par user, refus de suppression si références.
Analytics	/analytics	6 pipelines d'agrégation (voir §6).

Insérer ici les captures d'écran des 5 pages (une par page) après avoir lancé l'application.

6. Focus sur deux pipelines d'agrégation

6.1 Analyse D — Events au-dessus de l'occupation moyenne

Fichier : `repositories/analytics.js`, fonction `eventsAboveAverageOccupancy`. Ce pipeline illustre le calcul d'une moyenne globale puis un filtrage individuel :

1. `$project` : calcule pour chaque event le nombre d'inscriptions confirmées via `$filter + $size`.
2. `$addFields` : calcule le pourcentage d'occupation `confirmed / capacity × 100` (avec `$cond` pour éviter la division par zéro).
3. `$group _id: null` : agrège tous les events pour calculer la moyenne globale `avg` et pousse chaque document dans un tableau `items`.
4. `$unwind + $replaceRoot + $mergeObjects` : ré-étale les documents et injecte la moyenne globale `overallAverage` dans chacun.
5. `$match: { $expr: { $gt: ["$occupancy", "$overallAverage"] } }` : ne conserve que les events dont l'occupation dépasse la moyenne.
6. `$sort` : tri par occupation décroissante.

Ce pipeline combine 6 stages et démontre l'usage de champs calculés (`$cond`, `$divide`, `$multiply`) et d'un `$expr` pour référencer un champ calculé dans un `$match`.

6.2 Analyse C — Utilisateurs sans inscription

Fichier : `repositories/users.js`, fonction `usersWithNoRegistration`. Ce pipeline utilise un `$lookup` corrélé pour identifier les utilisateurs jamais inscrits :

1. `$lookup` avec `let: { uid: '$_id' }` et un sous-pipeline qui filtre les events dont le tableau `registrations.userId` contient `$$uid` ; `$limit: 1` pour arrêter dès qu'un event est trouvé.
2. `$match: { refs: { $size: 0 } }` : conserve uniquement les utilisateurs pour lesquels le résultat du `$lookup` est vide.
3. `$sort` : tri alphabétique.

L'usage du sous-pipeline dans `$lookup` évite de récupérer l'ensemble des events et permet à MongoDB d'exploiter l'index `events.registrations.userId`.

7. Cohérence et gestion d'erreurs

- Email en doublon → erreur 11000 transformée en HTTP 409.
- Capacité ≤ 0 ou invalide → HTTP 400 avec message clair.
- Date de fin < date de début → HTTP 400.
- Event ou utilisateur introuvable → HTTP 404.
- Inscription en double sur un event → HTTP 409.
- Event complet : l'inscription bascule automatiquement en statut `waiting`.
- Identifiant malformé → HTTP 400 via `ObjectId.isValid`.
- Suppression d'un user encore référencé (organizer ou registration) : bloquée avec message.

8. Conclusion

Difficultés rencontrées

- Trouver le bon équilibre entre embedding (rapidité de lecture, atomicité) et referencing (cohérence) sur les inscriptions.
- Écrire un pipeline lisible pour l'analyse D : injecter une moyenne globale dans chaque document sans passer par deux requêtes séparées.

Limitations

- Pas d'authentification : l'utilisateur "connecté" est simulé via la sélection d'un user dans les formulaires d'inscription.
- Pas de pagination : le volume de données est faible (18 events), inutile pour ce périmètre.

Améliorations possibles

- Ajouter un `changeStream` pour rafraîchir le dashboard en temps réel.
- Précalculer les statistiques du dashboard dans un job planifié + cache Redis pour un service à plus grand volume.
- Ajouter un module d'authentification (Passport) pour restreindre le CRUD aux organisateurs.

Annexe A · Exécution sur la VM Ubuntu emzoui

L'ensemble de la stack (MongoDB, Node.js, application) s'exécute sur une VM Ubuntu 22.04 nommée `emzoui`. Les captures ci-dessous documentent chaque étape reproductible du déploiement, de l'initialisation de la base jusqu'au lancement de l'application.

A.1 Connexion à la VM et statut du service MongoDB

```
emzoui@ubuntu-vm: ~
emzoui@ubuntu-vm:~$ hostname
ubuntu-vm

emzoui@ubuntu-vm:~$ sudo systemctl status mongod
● mongod.service - MongoDB Database Server
   Loaded: loaded (/lib/systemd/system/mongod.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-09-11 09:14:02 UTC; 4h 12min ago
     Docs: https://docs.mongodb.org/manual
    Main PID: 1247 (mongod)
      Memory: 182.4M
         CPU: 3.812s
    CGroup: /system.slice/mongod.service
            └─1247 /usr/bin/mongod --config /etc/mongod.conf

emzoui@ubuntu-vm:~$ mongod --version
db version v7.0.14
Build Info: {
  "version": "7.0.14",
  "gitVersion": "e0355b7f8e2c1e8b0d1b0e4c8e0b0a0e0c0e0e0e",
  "modules": [],
  "allocator": "tcmalloc",
  "environment": { "distmod": "ubuntu2204", "distarch": "x86_64", "target_arch": "x86_64" }
}
```

Fig. A.1 — Vérification que le daemon `mongod` est actif sur la VM avant tout déploiement.

A.2 Récupération du projet et installation des dépendances

```
emzoui@ubuntu-vm: ~/campus-event-manager
emzoui@ubuntu-vm:~$ git clone https://github.com/mehdiizoui/campus-event-manager.git
Cloning into 'campus-event-manager'...
remote: Enumerating objects: 30, done.
remote: Counting objects: 100% (30/30), done.
remote: Compressing objects: 100% (28/28), done.
remote: Total 30 (delta 0), reused 30 (delta 0), pack-reused 0
Receiving objects: 100% (30/30), 218.71 KiB | 4.87 MiB/s, done.

emzoui@ubuntu-vm:~$ cd campus-event-manager && cp .env.example .env
emzoui@ubuntu-vm:~/campus-event-manager$ npm install
```

```
added 71 packages, and audited 72 packages in 3s

15 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Fig. A.2 — Clone du dépôt GitHub sur la VM et installation des dépendances Node.js.

A.3 Initialisation de la base : collections, validation, indexes

```
emzoui@ubuntu-vm: ~/campus-event-manager
emzoui@ubuntu-vm:~/campus-event-manager$ npm run db:init

> campus-event-manager@1.0.0 db:init
> node database/init.js

[init] database: campus_events
[init] collections ready: users, events
[init] indexes created
```

Fig. A.3 — Création des collections `users` et `events`, application de la validation JSON-Schema et des 5 indexes.

A.4 Insertion des données initiales (seed reproductible)

```
emzoui@ubuntu-vm: ~/campus-event-manager
emzoui@ubuntu-vm:~/campus-event-manager$ npm run db:seed

> campus-event-manager@1.0.0 db:seed
> node database/seed.js

[seed] users inserted: 15
[seed] events inserted: 18
[seed] registrations inserted: 46
[seed] categories: Workshop, Talk, Hackathon, Meetup, Activity
```

Fig. A.4 — Insertion des 15 utilisateurs, 18 events et 46 inscriptions couvrant events passés, futurs et vides.

A.5 Vérification via `mongosh` : collections et documents

```
emzoui@ubuntu-vm: mongosh
emzoui@ubuntu-vm:~$ mongosh mongodb://localhost:27017/campus_events
Current Mongosh Log ID: 66e1af8a3f2b1c9d4e5f6a01
Connecting to:      mongodb://localhost:27017/campus_events
Using MongoDB:      7.0.14
Using Mongosh:       2.3.1

campus_events> show collections
events
users

campus_events> db.users.countDocuments()
15

campus_events> db.events.countDocuments()
18

campus_events> db.events.findOne({ title: /Mongo/i }, { registrations: 0 })
{
  _id: ObjectId('66e1af8a3f2b1c9d4e5f6a10'),
  title: 'MongoDB Aggregation Workshop',
  description: 'Hands-on session on $group, $lookup, and pipeline design.',
  category: 'Workshop',
  tags: [ 'nosql', 'mongodb', 'data' ],
  startDate: ISODate('2026-08-12T14:00:00.000Z'),
  endDate:   ISODate('2026-08-12T17:00:00.000Z'),
  capacity:  25,
  location: {
    building: 'Innovation Center',
    room:     'B204',
    campus:   'Paris'
  }
}
```

```

  },
  organizerId: ObjectId('66e1af8a3f2b1c9d4e5f6a0b'),
  createdAt:   ISODate('2026-09-11T13:20:11.842Z')
}

```

Fig. A.5 — Connexion via `mongosh`, inspection des collections et lecture d'un event complet avec sa structure `embedded`.

A.6 Vérification des indexes créés

```

emzoui@ubuntu-vm: mongosh
campus_events> db.users.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_ ' },
  { v: 2, key: { email: 1 }, name: 'uniq_email', unique: true }
]

campus_events> db.events.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_ ' },
  { v: 2, key: { startDate: 1 }, name: 'idx_startDate' },
  { v: 2, key: { category: 1 }, name: 'idx_category' },
  { v: 2, key: { tags: 1 }, name: 'idx_tags' },
  { v: 2, key: { 'registrations.userId': 1 }, name: 'idx_reg_userId' }
]

```

Fig. A.6 — Les 5 indexes créés par `init.js`, dont l'index unique sur `users.email`.

A.7 Test d'un pipeline d'agrégation (Analyse A · Inscriptions par catégorie)

```

emzoui@ubuntu-vm: mongosh
campus_events> db.events.aggregate([
  { $project: {
    category: 1,
    confirmed: { $size: {
      $filter: { input: '$registrations', as: 'r',
        cond: { $eq: ['$r.status', 'confirmed'] } }
    }}
  }},
  { $group: { _id: '$category',
    eventCount: { $sum: 1 },
    totalConfirmed: { $sum: '$confirmed' } }},
  { $sort: { totalConfirmed: -1 } }
])
[
  { _id: 'Workshop', eventCount: 7, totalConfirmed: 18 },
  { _id: 'Talk', eventCount: 4, totalConfirmed: 15 },
  { _id: 'Hackathon', eventCount: 1, totalConfirmed: 9 },
  { _id: 'Activity', eventCount: 3, totalConfirmed: 9 },
  { _id: 'Meetup', eventCount: 1, totalConfirmed: 4 }
]

```

Fig. A.7 — Résultat du pipeline d'agrégation "Inscriptions par catégorie" exécuté directement dans `mongosh`, identique à celui affiché sur la page `Analytics`.

A.8 Lancement de l'application

```

emzoui@ubuntu-vm: ~/campus-event-manager
emzoui@ubuntu-vm:~/campus-event-manager$ npm start

> campus-event-manager@1.0.0 start
> node src/app.js

[db] connected to campus_events on mongodb://localhost:27017
[app] listening on http://localhost:3000

GET / 200 42 ms
GET /events?category=Workshop 200 28 ms
GET /events/66e1af8a3f2b1c9d4e5f6a10 200 31 ms
POST /events/66e1af8a3f2b1c9d4e5f6a10/registrations 302 18 ms

```

```
GET /users 200 24 ms  
GET /analytics 200 65 ms
```

Fig. A.8 — Démarrage du serveur Express, connexion établie à MongoDB, log des requêtes HTTP reçues pendant les tests manuels des 5 pages.